

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\`a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-ad6f103249f55b1b5.jsonl`  
- SHA-256: `3b786870ca40294926310ff4a4468a0396535c4889f4963b6a6e3bbe98209d69`  
- Source modified: `2026-05-31T04:18:17+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_076beba8-59c`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

Transcript

1. user (2026-05-31T04:17:40.134Z)

Source Extractor

Research question: "Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):

1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMV3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:

- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly."

Fetch and extract key claims from this source:

****URL:**** <https://github.com/opedatalab/MinerU/blob/master/LICENSE.md>

****Title:**** opendatalab/MinerU ? LICENSE.md (moved AGPLv3 ? 'MinerU Open Source License' based on Apache-2.0)

****Found via:**** OCR + table-structure recognition pipelines (practitioner/implementation) search

Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
 - be a concrete, checkable statement (not vague generalities)
 - include a direct quote from the source as support
 - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-31T04:17:42.625Z)

I'll fetch the MinerU LICENSE.md page and extract the relevant claims.

3. user (2026-05-31T04:17:51.465Z)

MinerU License Terms

****Base License:**** Apache License 2.0

****Current Distribution:**** MinerU is licensed under Apache License 2.0 with additional proprietary terms. There is no indication in this document that it was previously under AGPLv3.

****Key Additional Restrictions:****

1. ****Commercial Thresholds:**** A separate commercial license is required if you and affiliates reach either threshold: "monthly active users (MAU) exceed 100 million" or "total monthly revenue exceeds USD 20 million."
2. ****Attribution Obligation:**** "If you provide online services to third parties based on MinerU, you must clearly and prominently indicate...that MinerU is used."

3. ****Automatic Termination:**** The license "will terminate automatically" if commercial licensing requirements aren't met or attribution obligations aren't fulfilled.

The document provides both English and Chinese versions with identical terms. The full Apache 2.0 text is referenced at <https://www.apache.org/licenses/LICENSE-2.0>.

4. assistant (2026-05-31T04:17:54.174Z)

The fetch succeeded. Let me note that the WebFetch summary may have missed the AGPLv3 history mentioned in the title. Let me verify the specific transition claim by fetching the raw file directly to confirm the exact wording.

5. user (2026-05-31T04:17:59.522Z)

MinerU License Analysis

****License Name & Opening:****

The license is called the "MinerU Open Source License." It opens with: "MinerU is licensed under Apache License 2.0 and is subject to the additional terms below."

****Relationship to Apache 2.0:****

The document states it is "licensed under" Apache License 2.0, not explicitly "based on" or "derived from."

****Commercial License Thresholds:****

- "monthly active users (MAU) exceed 100 million; or"
- "total monthly revenue exceeds USD 20 million"

****Attribution Requirement:****

"If you provide online services to third parties based on MinerU, you must clearly and prominently indicate, in the relevant product or service interface or in publicly available documentation, that MinerU is used."

****AGPL/Copyleft Mention:****

There is no mention of AGPL, AGPLv3, or copyleft/network-use clauses in this document.

6. assistant (2026-05-31T04:18:04.826Z)

I have the verified content. The page is the canonical license file in the MinerU repository (a primary source ? the project's own license document).

Key findings:

- MinerU is now under the "MinerU Open Source License," which is Apache-2.0 plus additional terms.
- It is no longer AGPLv3 (the title's claim about the AGPLv3 ? new-license transition is the framing; the current file itself shows Apache-2.0-based terms, no AGPL).
- Commercial use is permissive up to large thresholds (100M MAU or USD 20M monthly revenue), so it is commercially safe for muneem's single-user personal use.
- An attribution requirement applies only to providing online services to third parties ? not relevant to a local personal tool.

This is highly relevant to the research question's hard constraint #3 (commercial-license safety, verify weights vs code license).

7. user (2026-05-31T04:18:11.487Z)

Structured output provided successfully

8. assistant (2026-05-31T04:18:17.559Z)

Done. I fetched the MinerU LICENSE.md (primary source ? the project's own canonical license file at github.com/opendatalab/MinerU) and verified it against the raw file.

Key takeaway for muneem's license-safety constraint: MinerU is now under the "MinerU Open Source License" = Apache-2.0 + additional terms (not AGPLv3). The added terms only require a commercial license above 100M MAU or USD 20M/month revenue, and only impose attribution when offering online services to third parties ? neither applies to a single-user local personal-finance tool. No copyleft/network-use clauses remain. So for muneem's use case, MinerU's code is effectively commercial-license-safe.

One caveat worth flagging for the broader research (outside this source's scope): this is the MinerU/PDF-Extract-Kit code license only. The underlying model weights it pulls (layout/formula/OCR sub-models such as those from PDF-Extract-Kit) may carry separate licenses that should be verified independently.